UNIT - 03

LEAF AND NON-LEAF NODE COUNTING (Modified)

Problem Statement:

Develop a program in C++ that builds a binary search tree and counts its leaf and non-leaf nodes. The program is modified to display the actual leaf and non-leaf node values, to count the nodes at each level of the tree, and to let the user query the children of a selected node. The tree structure is not changed during counting; all results are checked against the inorder traversal.

Objective:

The objective of this project is to implement a binary search tree in C++ and count the leaf and non-leaf nodes using traversal, without modifying the tree structure. It also displays the actual leaf and non-leaf node values, counts the nodes at each level, and lets the user query the children of any selected node, while verifying all results against the inorder traversal sequence, and strengthening understanding of BST insertion, searching and traversal.

Algorithm:

Step 1: Start. Create an empty binary search tree (root = NULL) and an empty queue for level-order traversal.

Step 2: Display the menu: insert, count leaf and non-leaf nodes, display leaf and non-leaf values, count nodes at each level, query children, inorder display, exit.

Step 3: Read the choice and call the matching function. Repeat Steps 2 and 3 until choice = 0.

Step 4: Insert: Read the value. Start at the root; move left if the value is smaller and right if it is larger. If the value already exists, display "Duplicate value" and stop.

Step 5: When a NULL position is reached, create a new node with left = right = NULL and attach it there.

Step 6: Count leaf and non-leaf nodes: If the tree is empty, display "Tree is empty" and stop. Traverse every node of the tree.

Step 7: A node with left = NULL and right = NULL is a leaf node, so increase leaf count. Otherwise it is a non-leaf node, so increase non-leaf count. Display both counts.

Step 8: Display values: During the inorder traversal (left, node, right) store each leaf node value in the leaf list and each non-leaf node value in the non-leaf list.

Step 9: Print the leaf list and the non-leaf list. Both appear in ascending order because the traversal is inorder.

Step 10: Count nodes at each level: Insert the root into a queue and set level = 0.

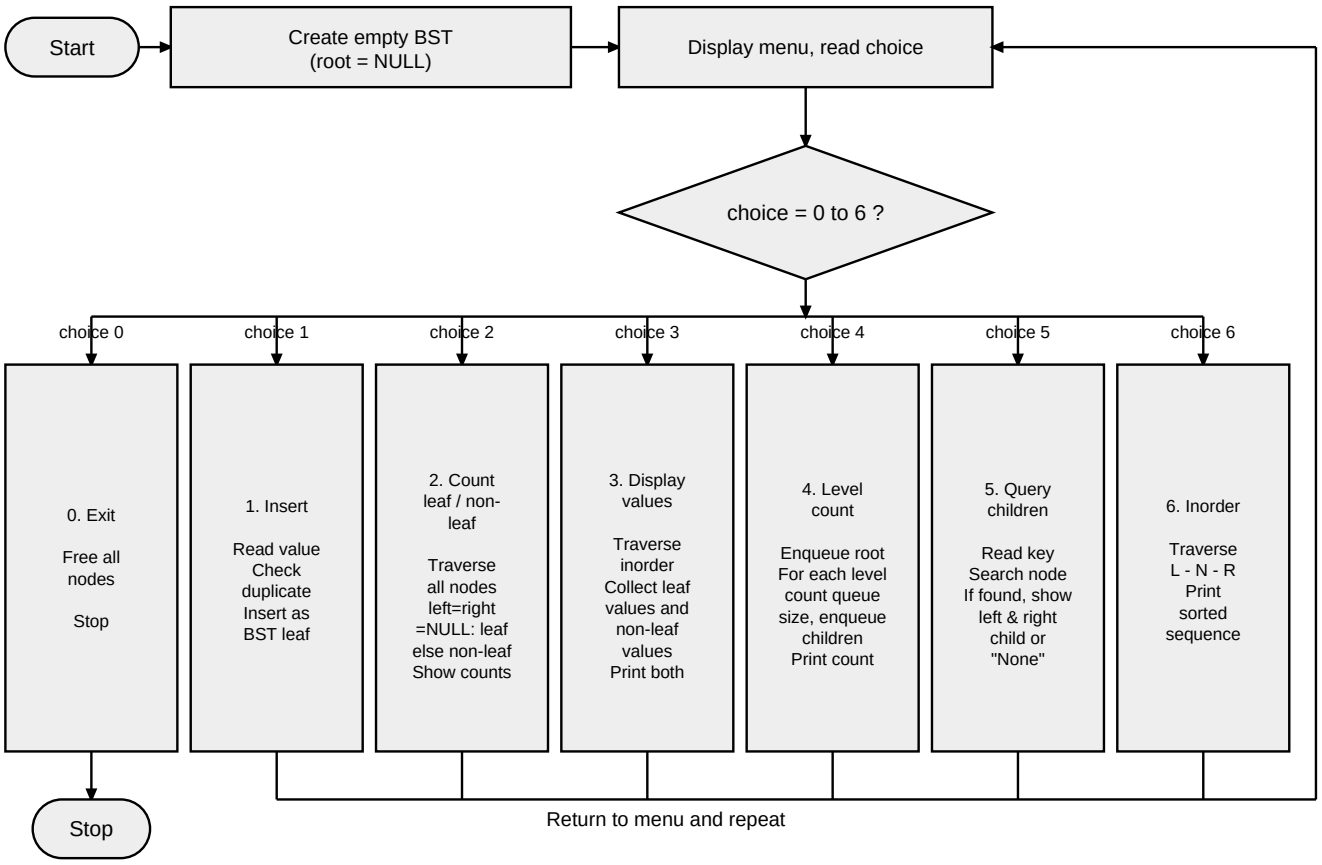
Step 11: Repeat while the queue is not empty: set count = current queue size. Remove count nodes one by one and insert their non-NULL left and right children into the queue.

Step 12: Display "Level (level): (count) node(s)" and increase level by 1. The sum of all level counts must equal the total node count.

Step 13: Query children: Read the key of the selected node and search for it from the root using the BST rule (go left if smaller, right if larger).

Step 14: If the node is not found, display "Node not found". Otherwise display its left child and right child (or "None" if a child is NULL) and state whether the node is a leaf or non-leaf.

Step 15: Inorder display: Print the inorder sequence and use it to verify that the leaf and non-leaf values and the counts are correct.

Step 16: Exit: Free all nodes of the tree and Stop.

Flowchart: Leaf and Non-Leaf Node Counting